

astrid-runtime/astrid

Astrid : un runtime de sécurité Rust très outillé en CI, mais 42 bugs ouverts, un seul mainteneur et des tests natifs difficiles à rejouer localement

Astrid is a portable, capability-secure operating system for composable software.

39

FICHIERS DE TEST
cargo test, insta

1

WORKFLOWS CI

42

BUGS ÉTIQUETÉS

192

ISSUES OUVERTES

<https://github.com/astrid-runtime/astrid> Licence Apache-2.0 Onboard · profil qa

☑ Que fait ce projet et quels sont ses parcours critiques ?

VERDICT

Les mécanismes de sécurité sont indépendants et « fail closed » : chaque parcours CLI se teste d'abord en négatif (accès refusé attendu), sur macOS et Linux seulement, Windows étant hors des archives de la v2026.9.0.

Astrid est « a portable, capability-secure operating system for composable software » : un runtime user-space sur macOS et Linux qui exécute des composants WebAssembly (« capsules ») dans un sandbox Wasmtime, sans autorité ambiante, chaque accès fichier/réseau/processus étant contrôlé par un jeton de capacité signé. Projet jeune (créé le 2026-02-15), 10277 étoiles, 216 issues ouvertes, dernier push le 2026-09-09, version courante v2026.9.0.

Parcours critiques à couvrir, tirés du README :

- 1 Cycle de vie du daemon : `astrid start / status / capsule list / stop`, avec retrait de l'état dans `astrid.volume` à l'arrêt et restauration au redémarrage.
- 2 Initialisation d'une distribution : `astrid init --distro <source>` (repo, manifeste local, bundle signé `--offline`), écriture d'un `Distro.lock` épinglant chaque capsule par hash BLAKE3.
- 3 Cycle de vie des capsules à chaud : `capsule new / build / install / update / remove` sans redémarrage.
- 4 Isolation par principal : `agent create`, `caps show`, `quota set`, `pair-device issue --scope use-only` ; un principal ne doit jamais lire l'espace d'un autre, et le système « fails closed ».
- 5 Montage du stockage durable : `storage mount / status / sync / unmount` via FSKit (macOS 26+) ou FUSE (Linux).

Chaque parcours traverse les mécanismes de sécurité indépendants (sandbox WASM, manifeste allow-list, ACL IPC, jeton de capacité, gate d'approbation, sandbox OS, chaîne d'audit) : ce sont eux qu'il faut tester en négatif. Windows est testé en CI mais absent des archives de la release.

☑ Comment est-il testé aujourd'hui ?

VERDICT

39 fichiers de test relevés entre `crates/astrid-integration-tests`, `crates/astrid-test` **et** `e2e/` (`cargo test + snapshots insta`), **mais aucun taux de couverture dans les données relevées.**

☐	<code>.cargo/</code>	config Cargo locale
☐	<code>.gemini/</code>	config agent Gemini
☐	<code>.github/</code>	20 workflows GitHub Actions
☐	<code>changes/</code>	notes de changement en attente
☐	<code>container/</code>	définitions de conteneur
☐	<code>crates/</code>	code principal : 33 crates Rust, 32 membres du workspace
☐	<code>docs/</code>	documentation
☐	<code>e2e/</code>	scénarios TOML et fixtures bout en bout
☐	<code>fuzz/</code>	tests de fuzzing
☐	<code>native/</code>	code natif macOS
☐	<code>release/</code>	config de release (nightly.toml)
☐	<code>scripts/</code>	scripts build, CI, release
☐	<code>.dockerignore</code>	exclusions du build Docker
☐	<code>.gitattributes</code>	attributs git
☐	<code>.gitignore</code>	exclusions git
☐	<code>.gitmodules</code>	sous-modules git
☐	... et 12 autres entrées à la racine du dépôt	

Pas de dossier `test/`, `tests/` ou `spec/` à la racine : les tests vivent sous `crates/` et `e2e/`. Le crate dédié est `crates/astrid-integration-tests`, 39 fichiers relevés, avec `cargo test` et des snapshots `insta` ; un crate `astrid-test` fournit `harness.rs` et `mocks.rs`.

Bout en bout : `e2e/` contient des scénarios TOML (`capability-scenarios.toml`, `cli-scenarios.toml`, `http-scenarios.toml`, `runtime-scenario-specs.toml`, `waiter-surfaces.toml`, `first-party-capsule-scenarios.toml`), un `concurrency.sh` et des fixtures de capsules adverses et concurrentes. Un dossier `fuzz/` existe à la racine.

Commande locale : `cargo test --workspace -- --quiet` ; en CI, `cargo test --workspace --locked` sur ubuntu et `scripts/ci/test-workspace-macos.sh` sur macos.

Règle d'équipe : « Tests required. New features need tests. Bug fixes need a regression test ».

Données non relevées : taux de couverture, nombre de cas dans les scénarios TOML, contenu de `runtime-e2e.yml` (« non lu ») ; à vérifier dans <https://github.com/astrid-runtime/astrid/tree/main/e2e> et <https://github.com/astrid-runtime/astrid/blob/main/github/workflows/runtime-e2e.yml>.

☑ Que dit la CI et quand tourne-t-elle ?

VERDICT

Un seul workflow lu, CI, sur push et pull_request, 14 jobs dont les tests sur ubuntu et macos ; les 19 autres workflows et l'état des runs restent à vérifier sur GitHub.

Un workflow principal, CI (.github/workflows/ci.yml), déclenché sur push et pull_request. Le push ne déclenche que sur main et stream-* et seulement si des .rs, Cargo.toml / Cargo.lock, des scripts de release ou certains workflows changent ; toute PR déclenche sans filtre (« Required build/test checks must also run for stacked and docs-only PRs »).

14 jobs, toolchain Rust 1.95 épinglée, harden-runner sur chaque job :

- ❑ Qualité : fmt, clippy -D warnings, check --all-features, msrv (1.95), audit (cargo-audit 0.22.2).
- ❑ Tests : test (ubuntu + macos, CARGO_PROFILE_TEST_DEBUG=0, cf. #954), linux-fuse-e2e (un test --ignored de montage FUSE natif), linux-v0104-upgrade (script de migration depuis v0.10.4), windows-filesystem-native (WinFsp 2.1, x86_64 et aarch64), wasm-portability.
- ❑ Release : linux-release-smoke (4 cibles gnu/musl, contrôles glibc ≤ 2.34 et ELF statique).
- ❑ PR seulement, consultatifs : api-compatibility (cargo-semver-checks, cf. #1480), public-rust-api-diff, gateway-openapi-contract.

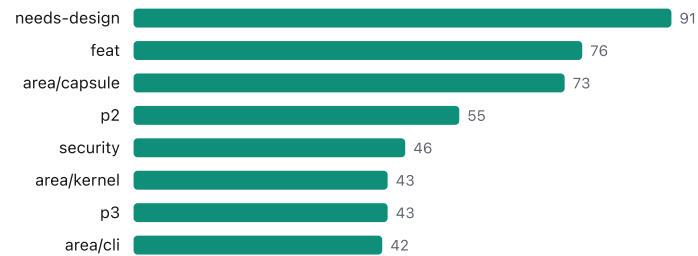
Au total 20 workflows dans .github/, dont runtime-e2e.yml, native-storage-certification.yml, supervised-storage-certification.yml, storage-benchmark.yml, windows-local-transport.yml, codeql.yml, nightly.yml. Le risque CI est évalué « low ».

Données non relevées : l'état des derniers runs et les déclencheurs des 19 autres workflows ; à vérifier dans <https://github.com/astrid-runtime/astrid/actions>.

☑ Où sont les bugs connus ?

VERDICT

42 issues bug sur 192 ouvertes, 36 en p1 , concentrées sur area/capsule (73) et area/kernel (43) ; **91 issues** needs-design n'ont pas encore de comportement à tester.



192 issues ouvertes, 149 fermées sur 30 jours. Étiquettes utiles pour un plan de test : bug 42, security 46, test 16, blocked 16 ; priorités p1 36, p2 55, p3 43.

Zones les plus chargées : area/capsule 73, area/kernel 43, area/cli 42, area/runtime 25, area/core 25. 91 issues sont needs-design : comportement non figé, donc cas de test à écrire avec prudence.

Bugs concrets et reproductibles parmi les « good first » : #459 « Markdown parser in TUI fails on numbered lists >= 10 », #276 « MCP interceptor: propagate tool_result.is_error instead of treating it as success », #461 « is_known_tag coverage table is 18 vs 19 and omits ToolCancelRequest », #476 sur l'enregistrement dupliqué dans CapsuleRegistry .

L'issue la plus discutée est un ticket de test : #1708 « test(container): qualify the signed amd64 Linux-backed Astrid host profile », 29 commentaires.

Données non relevées : le croisement bug × area/* et la liste des 42 bugs ; à vérifier dans <https://github.com/astrid-runtime/astrid/issues?q=is%3Aopen+label%3Abug>.

☑ Comment reproduire et signaler un bug ici ?

VERDICT

Pas d'issue sans reproduction (version, étapes, attendu et observé), pas de PR sans issue assignée ni test de régression, jamais de vulnérabilité en issue publique ; 24 PR ouvertes attendent déjà.

Procédure imposée par CONTRIBUTING.md :

- 1 Chercher une issue existante, puis utiliser le template « bug report ».
- 2 Reproduire avant d'ouvrir : version ou commit affecté, étapes exactes, comportement attendu et observé, preuves. Les audits en masse, findings spéculatifs et rapports générés par IA non vérifiés sont refusés (« AI output is a drafting aid, not evidence »).
- 3 Vulnérabilité : jamais d'issue publique, passer par GitHub Security Advisories ; une politique `SECURITY.md` existe.
- 4 Pour corriger soi-même : issue assignée d'abord (« Every PR must be linked to an issue. No exceptions »), régression obligatoire pour un bug fix, commit `git commit -s` (DCO), fragment `changes/{issue}.fixed.md` , titre Conventional Commits, template PR complet sinon rejet CI.

Le flux est actif : 24 PR ouvertes et une longue liste de correctifs fusionnés en 30 jours, presque tous en `fix(...)` , par exemple #1906 « fix(init): honor explicit grants after completed batch resume » et #1870 « fix(test): provision private Windows run directory ». Une PR de test récente donne le modèle d'une fixture négative : #1849 « test(capsule): use an unsatisfiable astrid-version fixture ».

Donnée non relevée : le contenu des templates d'issue (`.github/ISSUE_TEMPLATE`) ; à vérifier dans <https://github.com/astrid-runtime/astrid/tree/main/.github>.

☑ Quelles zones sont peu couvertes ?

VERDICT

Sans mesure de couverture, les zones à risque se déduisent : bus factor 1 (309 commits pour `joshuajbouw`), Windows hors release, montage FUSE réduit à un test `--ignored` , 16 issues `test` et 16 `blocked` .

Donnée non relevée : la couverture de code ; à vérifier dans <https://github.com/astrid-runtime/astrid/actions> (job `test`). Les signaux ci-dessous sont indirects.

- ❑ Concentration humaine : bus factor 1, `joshuajbouw` 309 commits contre 7 pour le second contributeur ; risque « high ». Le relecteur et l'auteur des tests sont la même personne : la revue QA externe a de la valeur.
- ❑ Rythme soutenu sans creux : 41 commits en W26, 17 en W37, dernier commit le 2026-09-09 : les régressions arrivent vite, rejouer les scénarios `e2e/` à chaque release.
- ❑ Plateformes : Windows testé en CI mais hors release ; le montage FUSE natif est un seul test `--ignored` en CI ; FSKit exige macOS 26+ et une approbation d'extension signée, difficile à automatiser.
- ❑ Dépendances : 117 entrées dans `Cargo.toml` , 90 externes, 8 épinglées en exact ; les deux PR Dependabot ouvertes (#1834, #1877) sont des candidats à une passe de non-régression.
- ❑ Fonctionnel : 16 issues `test` et 16 `blocked` ; les thèmes chauds `campaign/os-universal` (32 issues) et les décisions en attente (#1664 « specify the first-owner enrollment ceremony », #1692) n'ont pas encore de comportement spécifié à tester.
- ❑ Le contenu de `fuzz/` et des 19 workflows hors `ci.yml` n'a pas été lu.



Plan de test en 5 points

- 1 Fumée sur macOS et Linux : `cargo test --workspace -- --quiet` , puis le cycle `astrid start / status / capsule list / stop` en vérifiant que l'état est retiré dans `astrid.volume` à l'arrêt et restauré au redémarrage.
- 2 Sécurité en négatif, risque haut : pour `astrid init --distro` , `capsule install` , `agent create` et `storage mount` , un cas sans jeton de capacité ou avec manifeste vide doit être refusé (« fails closed ») ; rejouer `e2e/capability-scenarios.toml` et les fixtures `e2e/fixtures/astrid-capsule-adversarial` .
- 3 Régression sur les 42 `bug` ouverts, en commençant par les reproductibles : #459 (listes numérotées ≥ 10 dans le TUI), #276 (`is_error` MCP traité comme un succès), #461 (table `is_known_tag` 18 vs 19).
- 4 Plateformes, risque moyen : un passage complet sur macOS 26+ avec FSKit (approbation d'extension signée) et un sur Linux FUSE, où la CI ne joue qu'un test `--ignored` (`linux-fuse-e2e`) ; Windows est testé en CI mais hors archives, à traiter comme non supporté.
- 5 Non-régression des dépendances et des mises à niveau : rejouer la suite sur les PR Dependabot #1834 et #1877, sur la migration depuis v0.10.4 (`linux-v0104-upgrade`), et suivre #1708 (29 commentaires) qui qualifie le profil hôte Linux.